УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«МОГИЛЕВСКИЙ ГОСУДАРСТВЕННЫЙ ПОЛИТЕХНИЧЕСКИЙ КОЛЛЕДЖ»

Специальность 5-04-0612-02

Учебный предмет Инструментальное программное обеспечение

УТВЕРЖДАЮ
Заместитель директора
по учебной работе
_____М.М.Федоськова
_____

ЛАБОРАТОРНАЯ РАБОТА № 6

СОЗДАНИЕ ПРОСТЕЙШИХ ПРОГРАММ ОБРАБОТКИ ОДНОМЕРНЫХ МАССИВОВ (СПИСКОВ)

МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ

Разработал преподаватель
Денисовец Д.А.

2026

Обсуждено и одобрено
на заседании цикловой комиссии
специальностей в области
программного обеспечения
информационных систем
Протокол № ____от _____
Председатель цикловой комиссии
_____ Д.А.Денисовец

1 Цель работы

1.1 Создание простейших программ обработки одномерных массивов (списков)

2 Методическое и материальное обеспечение

2.1 Персональный компьютер
2.2 Методические рекомендации по выполнению лабораторной работы
2.3 Интегрированная среда разработки PyCharm / Visual Studio Code

3 Последовательность выполнения работы

3.1 Ознакомиться с основными теоретическими положениями
3.2 Выполнить индивидуальное задание
3.3 Оформить отчет
3.4 Ответить на контрольные вопросы

4 Основные теоретические положения

4.1 Общие сведения об типах данных

Списки, кортежи, множества и диапазоны – это нумерованные наборы объектов. Каждый элемент набора содержит лишь ссылку на объект – по этой причине они могут содержать объекты произвольного типа данных и иметь неограниченную степень вложенности. Позиция элемента в наборе задается индексом. Обратите внимание на то, что нумерация элементов начинается с 0, а не с 1.

Списки и кортежи являются просто упорядоченными последовательностями элементов. Как и все последовательности, они поддерживают обращение к элементу по индексу, получение среза, конкатенацию (оператор +), повторение (оператор *), проверку на вхождение (оператор in) и невхождение (оператор not in).

Списки относятся к изменяемым типам данных. Это означает, что можно не только получить элемент по индексу, но и изменить его:

```
arr = [1, 2, 3]  # Создаем список
print(arr[0]) # Получаем элемент по индексу
1
arr[0] = 50 # Изменяем элемент по индексу
print(arr)
[50, 2, 3]
```

Кортежи относятся к неизменяемым типам данных. Иными словами, можно получить элемент по индексу, но изменить его нельзя:

```
t = (1, 2, 3) # Создем кортеж
print(t[0]) # Получаем элемент по индексу
1
```

```
print(t[0] = 50) # Изменить элемент по индексу нельзя!
Traceback (most recent call last):
  File "<pyshell#2>", line 1, in <module>
    t[0] = 50 # Изменить элемент по индексу нельзя!
TypeError: 'tuple' object does not support item assignment
```

Множества могут быть как изменяемыми, так и неизменяемыми. Их основное отличие от только что рассмотренных типов данных - хранение лишь уникальных значений (неуникальные значения автоматически отбрасываются). Пример:

```
print(set([0, 1, 1, 2, 3, 3, 4]))
{0, 1, 2, 3, 4}
```

Что касается диапазонов, то они представляют собой наборы чисел, сформированные на основе заданных начального, конечного значений и величины шага между числами. Их важнейшее преимущество перед всеми остальными наборами объектов - небольшой объем занимаемой оперативной памяти. Пример:

```
r = range(0, 101, 10)
for i in r: print(i, end=" ")
0 10 20 30 40 50 60 70 80 90 100
```

4.2 Создание списка в Python

Создать список можно следующими способами:

- **с помощью функции list ([<Последовательность>])**. Функция позволяет преобразовать любую последовательность в список. Если параметр не указан, то создается пустой список.

Примеры:

```
print(list ()) # Создаем пустой список
[]
print(list ("String")) # Преобразуем строку в список
['S', 't', 'r', 'i', 'n', 'g']
print(list ( (1,  2,  3,  4,  5) ))  # Преобразуем кортеж в  список
[1,  2,  3,  4,  5]
```

- перечислив все элементы списка внутри квадратных скобок:

```
arr = [1, "str", 3, "4"]
print(arr)
[1, 'str', 3, '4']
```

- заполнив список поэлементно с помощью метода **append ()**:

```
arr = [ ] # Создаем пустой список
arr.append (1) # Добавляем элемент1 (индекс 0)
arr.append ("str" ) # Добавляем элемент2 (индекс 1)
print(arr)
[1, 'str']
```

В некоторых языках программирования (например, в **PHP**) можно добавить элемент, указав пустые квадратные скобки или индекс больше последнего индекса. В языке **Python** все эти способы приведут к ошибке:

```
arr = []
arr[] = 10
SyntaxError: invalid syntax
```

При создании списка в переменной сохраняется ссылка на объект, а не сам объект. Это обязательно следует учитывать при групповом присваивании. Групповое присваивание можно использовать для чисел и строк, но для списков этого делать нельзя. Рассмотрим пример:

```
x = y = [1,   2] # Якобы создали два объекта
print(x,   y)
([1, 2], [1, 2])
```

В этом примере необходимо создать список из двух элементов и присвоить значение переменным **x** и **y**. Теперь попробуем изменить значение в переменной **y**:

```
y[1]  = 100 # Изменяем второй элемент
print(x, y) # Изменилось значение сразу в двух переменных
([1, 100], [1, 100])
```

Как видно из примера, изменение значения в переменной **y** привело также к изменению значения в переменной **x**. Таким образом, обе переменные ссылаются на один и тот же объект, а не на два разных объекта. Чтобы получить два объекта, необходимо производить раздельное присваивание:

```
x, y = [1, 2], [1, 2]
y[1] = 100 # Изменяем второй элемент
print(x, y)
([1, 2], [1, 100])
```

Точно такая же ситуация возникает при использовании оператора повторения *. Например, в следующей инструкции производится попытка создания двух вложенных списков с помощью оператора *:

```
arr =[ [] ] * 2 # Якобы создали два вложенных списа
print(arr)
[[], []]
arr[0].append (5) # Добавляем элемент
print(arr) # Изменились два элемента
[[5], [5]]
```

Создавать вложенные списки следует с помощью метода **append ()** внутри цикла:

```
arr = [ ]
for i in range (2): arr.append ([])
print(arr)
[ [], [] ]
arr [0].append(5)
print(arr)
[ [5], [] ]
```

Можно также воспользоваться генераторами списков:

```
arr = [ []   for  i  in range (2) ]
print(arr)
[ [], [] ]
```

Проверить, ссылаются ли две переменные на один и тот же объект, позволяет оператор **is**. Если переменные ссылаются на один и тот же объект, то оператор **is** возвращает значение **True**:

```
x = y = [1, 2]   # Неправильно
print(x is y) # Переменные содержат ссыпку на один и тот же список
True
x, y = [1, 2], [1, 2] # Правильно
print(x is y)        # Это разные объекты
False
```

Но что же делать, если необходимо создать копию списка? Первый способ заключается в применении операции извлечения среза, второй - в использовании функции **list ()**, а третий - в применении появившегося в **Python 3.3** метода **copy ()**.

```
x = [1, 2, 3, 4, 5] # Создали список
# Создаем копию списка
y = list(x) # или с помощью среза: y = x[:]
            # или вызовом метода copy(): y = x.copy()
print(y)
[1, 2, 3, 4, 5]
print(x is y) # Оператор показывает, что это разные объекты
False
y[1] = 100 # Изменяем второй элемент
print(x, y)
([1, 2, 3, 4, 5], [1, 100, 3, 4, 5])
```

На первый взгляд может показаться, что получить копию – оператор **is** показывает, что это разные объекты, а изменение элемента затронуло лишь значение переменной **y**. В данном случае вроде все нормально. Но проблема заключается в том, что списки в языке **Python** могут иметь неограниченную степень вложенности.

Рассмотрим это на примере:

```
x = [1, [2, 3, 4, 5]] # Создали вложенный список
y = list(x) # Якобы сделали копию списка
print(x is y) # Разные объекты
False
y[1][1] = 100 # Изменяем элемент
print(x, y) # Изменение затронуло переменную x!
([1, [2, 100, 4, 5]], [1, [2, 100, 4, 5]])
```

Создали список, в котором второй элемент является вложенным списком. Далее с помощью функции **list ()** попытались создать копию списка. Как и в предыдущем примере, оператор **is** показывает, что это разные объекты, но посмотрите на результат. Изменение переменной **y** затронуло и значение переменной **x**. Таким образом, функция **list ()** и операция извлечения среза создают лишь **поверхностную копию списка**.

Чтобы получить полную копию списка, следует воспользоваться функцией **deepcopy()** из модуля **copy**:

```
import copy # Подключаем модуль copy
```

```
x = [1, [2, 3, 4, 5]]
y = copy.deepcopy(x) # Делаем полную копию списка
y[1][1] = 100 # Изменяем второй элемент
print(x, y) # Изменился только список в переменной y
([1, [2, 3, 4, 5]], [1, [2, 100, 4, 5]])
```

Функция **deepcopy ()** создает копию каждого объекта, при этом сохраняя внутреннюю структуру списка. Иными словами, если в списке существуют два элемента, ссылающиеся на один объект, то будет создана копия объекта, и элементы будут ссылаться на этот новый объект, а не на разные объекты.

Пример:

```
import copy # Подключаем модуль copy
x = [1, 2]
y = [x, x] # Два элемента ссылаются на один объект
print(y)
[[1, 2], [1, 2]]
z = copy.deepcopy(y) # Сделали копию списка
print(z[0] is x, z[1] is x, z[0] is z[1])
(False, False, True)
z[0][0] = 300 # Изменили один элемент
print(z) # Значение изменилось сразу в двух элементах!
[[300, 2], [300, 2]]
print(x) # начальный список не изменился
[1, 2]
```

4.3 Операции над списками в Python

Обращение к элементам списка осуществляется с помощью квадратных скобок, в которых указывается индекс элемента. Нумерация элементов списка начинается с нуля. Выведем все элементы списка:

```
arr = [1, "str", 3.2, "4"]
print(arr[0], arr[1], arr[2], arr[3])
(1, 'str', 3.2, '4')
```

С помощью позиционного присваивания можно присвоить значения элементов списка каким-либо переменным. Количество элементов справа и слева от оператора = должно совпадать, иначе будет выведено сообщение об ошибке:

```
x, y, z = [1, 2, 3]   # Позиционное присваивание
print(x, y, z)
(1, 2, 3)
print(x, y = [1, 2, 3]) #  Количество элементов должно совпадать
Traceback (most recent call last):
  File "<pyshell#86>", line 1, in <module>
    x, y = [1, 2, 3] # Количество элементов должно  совпадать
ValueError:  too many values to unpack (expected 2)
```

В **Python 3** при позиционном присваивании перед одной из переменных слева от оператора = можно указать звездочку. В этой переменной будет

сохраняться список, состоящий из "лишних" элементов. Если таких элементов нет, то список будет пустым:

```
x, y, *z = [1, 2, 3]
print(x,   y,   z)
(1, 2, [3])
x, y, *z = [1, 2, 3, 4, 5]
print(x, y, z)
(1, 2, [3, 4, 5])
x, y, *z = [1, 2]
print(x, y, z)
(1, 2, [ ])
x, y, z = [1, 2]
print(x, y, z)
([], 1, 2)
x, *y, z = [1, 2, 3, 4, 5]
print(x, y, z)
(1, [2, 3, 4], 5)
*z, = [1, 2, 3, 4, 5]
print(z)
[1, 2, 3, 4, 5]
```

Так как нумерация элементов списка начинается с 0, индекс последнего элемента будет на единицу меньше количества элементов. Получить количество элементов списка позволяет функция **len()**:

```
arr = [1, 2, 3, 4, 5]
print(len(arr)) # Получаем количество элементов
5
print(arr[len(arr) - 1])  # Получаем последний элемент
5
```

Если элемент, соответствующий указанному индексу, отсутствует в списке, то возбуждается исключение **IndexError**:

```
arr = [1, 2, 3, 4, 5]
print(arr[5]) # Обращение к несуществующему элементу
Traceback (most recent call last) :
File  "<pyshell#99>", line 1, in <module>
  arr[5] # Обращение к несуществуоцему элементу
IndexError: list index out of range
```

В качестве индекса можно указать отрицательное значение. В этом случае смещение будет отсчитываться от конца списка, а точнее - чтобы получить положительный индекс, значение вычитается из общего количества элементов списка:

```
arr = [1, 2, 3, 4, 5]
print(arr[-1], arr[len(arr) -1]) # Обращение к последнему элементу
(5, 5)
```

Так как списки относятся к изменяемым типам данных, то можно изменить элемент по индексу:

```
arr = [1, 2, 3, 4, 5]
arr[0] = 600 # Изменение элемента по индексу
arr
[600, 2, 3, 4, 5]
```

Кроме того, списки поддерживают операцию извлечения среза, которая возвращает указанный фрагмент списка. Формат операции:

[<Начало>:<Конец>:<Шаг>]

Все параметры не являются обязательными. Если параметр **<Начало>** не указан, то используется значение 0. Если параметр **<Конец>** не указан, то возвращается фрагмент до конца списка. Следует также заметить, что элемент с индексом, указанном в этом параметре, не входит в возвращаемый фрагмент. Если параметр **<Шаг>** не указан, то используется значение 1. В качестве значения параметров можно указать отрицательные значения.

Теперь рассмотрим несколько примеров. Сначала получим поверхностную копию списка:

```
arr = [1, 2, 3, 4, 5]
m = arr[:]
print(m) # Создаем поверхностную копию к выводим значения
[1, 2, 3, 4, 5]
print(m is arr) # Оператор is показывает, что это разные объекты
False
```

Теперь выведем символы в обратном порядке:

```
arr = [1, 2, 3, 4, 5]
print(arr[: :-1]) # Шаг  -1
[5, 4, 3, 2, 1]
```

Выведем список без первого и последнего элементов:

```
print(arr [1:]) #  Без первого элемента
[2, 3, 4, 5]
print(arr [:-1]) # Без последнего элемента
[1, 2, 3, 4]
```

Получим первые два элемента списка:

```
print(arr[0:2]) # Символ с индексом 2 не входит в диапазон
[1, 2]
```

А теперь получим последний элемент:

```
print(arr[-1:]) #  Последний элемент  списка
[5]
```

И, наконец, выведем фрагмент от второго элемента до четвертого включительно:

```
print(arr [1:4]) #  Возвращаются элементы с индексами  1, 2  и 3
[2, 3, 4]
```

С помощью среза можно изменить фрагмент списка. Если срезу присвоить пустой список, то элементы, попавшие в срез, будут удалены:

```
arr = [1, 2, 3, 4, 5]
arr[1:3] = [6, 7] # Изменяем значения элементов с индексами 1 и 2
print(arr)
```

```
[1, 6, 7, 4, 5]
arr [1:3] = [ ]  # Удаляем элементы с индексами 1 и 2
print(arr)
[1, 4, 5]
```

Соединить два списка в один список позволяет оператор +. Результатом объединения будет новый список:

```
arr1 = [1, 2, 3, 4, 5]
arr2 = [6, 7, 8, 9]
arr3 = arr1 + arr2
print(arr3)
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Вместо оператора + можно использовать оператор +=. Следует учитывать, что в этом случае элементы добавляются в текущий список:

```
arr = [1, 2, 3, 4, 5]
arr += [6, 7, 8, 9]
print(arr)
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Кроме рассмотренных операций, списки поддерживают операцию повторения и проверку на вхождение. Повторить список указанное количество раз можно с помощью оператора *, а выполнить проверку на вхождение элемента в список позволяет оператор **in**:

```
print( [1, 2, 3] * 3 )# Операция повторения
[1, 2, 3, 1, 2, 3, 1, 2, 3]
print(2 in [1, 2, 3, 4, 5], 6 in [1, 2, 3, 4, 5]) # Проверка на вхождение
(True, False)
```

4.4 Многомерные списки в Python

Любой элемент списка может содержать объект произвольного типа. Например, элемент списка может быть числом, строкой, списком, кортежем, словарем и т. д. Создать вложенный список можно, например, так:

```
arr = [ [1, 2, 3], [4, 5, 6], [7, 8, 9] ]
```

Выражение внутри скобок может располагаться на нескольких строках. Следовательно, предыдущий пример можно записать иначе:

```
arr=[       [1,2,3],
            [4,5,6],
            [7,8,9]
        ]
```

Чтобы получить значение элемента во вложенном списке, следует указать два индекса:

```
print(arr[l] [1])
```

Результат выполнения:

```
5
```

Элементы вложенного списка также могут иметь элементы произвольного типа. Количество вложений не ограничено. То есть, можно создать объект любой

степени сложности. В этом случае для доступа к элементам указывается несколько индексов подряд. Примеры:

```
arr = [[1, ["a", "b"], 3], [4, 5, 6], [7, 8, 9]]
print(arr[0][1][0])
'a'
arr = [[1, {"a": 10, "b": ["s", 5]}]]
print(arr[0][1]["b"][0])
's'
```

4.5 Перебор элементов списка

Перебрать все элементы списка можно с помощью цикла **for**:

```
arr = [1, 2, 3, 4, 5]
for i in arr: print (i, end=" ")
1 2 3 4 5
```

Следует заметить, что переменную **i** внутри цикла можно изменить, но если она ссылается на неизменяемый тип данных (например, число или строку), то это не отразится на исходном списке:

```
arr = [1, 2, 3, 4] # Элементы имеют неизменяемый тип (число)
for i in arr: i += 10
print(arr) #  Список не изменился
[1, 2, 3, 4]
arr = [ [1, 2], [3, 4] ] # Элементы имеют изменяемый тип (список)
for i in arr: i[0] += 10
print (arr) #  Список изменился
[[11, 2], [13, 4]]
```

Чтобы получить доступ к каждому элементу, можно, например, для генерации индексов воспользоваться функцией **range ()**. Функция возвращает объект-диапазон, поддерживающий итерации, а с помощью диапазона внутри цикла **for** можно получить текущий индекс. Функция **range ()** имеет следующий формат:

```
range([<Начало>, ]<Конец>[, <Шаг>])
```

Первый параметр задает начальное значение. Если параметр **<Начало>** не указан, то по умолчанию используется значение 0. Во втором параметре указывается конечное значение. Следует заметить, что это значение не входит в возвращаемый диапазон значений. Если параметр **<Шаг>** не указан, то используется значение 1. Для примера умножим каждый элемент списка на 2:

```
arr = [1, 2, 3, 4]
for i in range(len(arr)):
    arr[i] *= 2
print(arr) # Результат выполнения: [2, 4, 6, 8]
```

Можно также воспользоваться функцией **enumerate (<Объект> [, start=0])**, которая на каждой итерации цикла **for** возвращает кортеж из индекса и значения текущего элемента списка. Умножим каждый элемент списка на 2:

```
arr = [1, 2, 3, 4]
```

```
for i, elem in enumerate(arr):
    arr[i] = elem * 2
print(arr) # Результат выполнения: [2, 4, 6, 8]
```

Кроме того, перебрать элементы можно с помощью цикла **while**. Но в этом случае следует помнить, что цикл **while** работает медленнее цикла **for**. Для примера умножим каждый элемент списка на 2, используя цикл **while**:

```
arr = [1, 2, 3, 4]
i, c = 0, len(arr)
while i < c:
    arr[i] *= 2
    i += 1
print (arr) #  Результат выполнения: [2, 4, 6, 8]
```

4.6 Сортировка списка в Python

Отсортировать список позволяет метод **sort (). Он имеет следующий формат:**

```
sort([key=None][, reverse=False])
```

Все параметры не являются обязательными. Метод изменяет текущий список и ничего не возвращает. Отсортируем список по возрастанию с параметрами по умолчанию:

```
arr = [2, 7, 10, 4, 6, 8, 9, 3, 1, 5]
arr.sort () # Изменяет текущий список
print(arr)
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Чтобы отсортировать список по убыванию, следует в параметре **reverse** указать значение **True**:

```
arr = [2, 7, 10, 4, 6, 8, 9, 3, 1, 5]
arr.sort (reverse=True) # Сортировка по убыванию
print(arr)
[10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
```

4.7 Добавление и удаление элементов списка в Python

Для добавления и удаления элементов списка используются следующие методы:

– **append (<Объект>)** - добавляет один объект в конец списка. Метод изменяет текущий список и ничего не возвращает.

Пример:

```
arr = [1, 2, 3]
arr.append (4)
print(arr) # Добавляем число
[1, 2, 3, 4]
arr.append ([5, 6])
print(arr) # Добавляем список
```

```
[1, 2, 3, 4, [5, 6]]
arr.append ( (7, 8) )
print(arr) # Добавляем кортеж
[1, 2, 3, 4, [5, 6], (7, 8)]
```

– **extend (<Последовательность>)** - добавляет элементы последовательности в конец списка. Метод изменяет текущий список и ничего не возвращает.

Пример:

```
arr = [1, 2, 3]
arr.extend( [4, 5, 6] )  # Добавляем список
arr.extend( (7, 8, 9) ) # Добавляем кортеж
arr.extend( "abc" ) # Добавляем буквы из строки
print(arr)
[1, 2, 3, 4, 5, 6, 7, 8, 9, 'a', 'b', 'c']
```

Добавить несколько элементов можно с помощью операции конкатенации или оператора +=:

```
arr = [1, 2, 3]
print(arr + [4, 5, 6]) # Возвращает новый список
[1, 2, 3, 4, 5, 6]
arr += [4, 5, 6] # Изменяет текущий список
print(arr)
[1, 2, 3, 4, 5, 6]
```

Кроме того, можно воспользоваться операцией присваивания значения срезу:

```
arr = [1, 2, 3]
arr [len(arr) : ] = [4, 5, 6] # Изменяет текущий список
print(arr)
[1, 2, 3, 4, 5, 6]
```

– **insert (<Индекс>, <Объект>)** - добавляет один объект в указанную позицию. Остальные элементы смещаются. Метод изменяет текущий список и ничего не возвращает.

Примеры:

```
arr = [1, 2, 3]
arr.insert (0, 0)
print(arr)  # Вставляем 0 в начало списка
[0, 1, 2, 3]
arr.insert (-1, 20)
print(arr) # Можно указать отрицательные числа
[0, 1, 2, 20, 3]
arr.insert (2, 100)
print(arr) # Вставляем 100 в позицию 2
[0, 1, 100, 2, 20, 3]
arr.insert (10, [4, 5])
print(arr) # Добавляем список
[0, 1, 100, 2, 20, 3, [4, 5]]
```

Метод **insert ()** позволяет добавить только один объект. Чтобы добавить несколько объектов, можно воспользоваться операцией присваивания значения срезу. Добавим несколько элементов в начало списка:

```
arr = [1, 2, 3]
arr[:0] = [-2, -1, 0]
print(arr)
[-2, -1, 0, 1, 2, 3]
```

– **pop ([<Индекс>])** - удаляет элемент, расположенный по указанному индексу, и возвращает его. Если индекс не указан, то удаляет и возвращает последний элемент списка. Если элемента с указанным индексом нет, или список пустой, возбуждается исключение **IndexError**.

Примеры:

```
arr = [1, 2, 3, 4, 5]
print(arr.pop())  # Удаляем последний элемент списка
5
print(arr) # Список изменился
[1, 2, 3, 4]
print(arr.pop(0))  # Удаляем первый элемент списка
1
print(arr) # Список изменился
[2, 3, 4]
```

Удалить элемент списка позволяет также оператор **del**:

```
arr = [1, 2, 3, 4, 5]
del arr [4]
print(arr) # Удаляем последний элемент списка
[1, 2, 3, 4]
del arr[:2]
print(arr) # Удаляем первый и второй элементы
[3, 4]
```

– **remove (<Значение>)** - удаляет первый элемент, содержащий указанное значение. Если элемент не найден, возбуждается исключение **ValueError**. Метод изменяет текущий список и ничего не возвращает.

Примеры:

```
arr = [1, 2, 3, 1, 1]
arr.remove (1) # Удаляет только первый элемент
print(arr)
[2, 3, 1, 1]
print(arr. remove (5)) #  Такого элемента  нет
Traceback  (most  recent  call  last):
File  "<pysheli#3>",  line  1,  in <module>
arr.remove (5) #  Такого элемента  нет
ValueError: list.remove(x):  x not in list
```

– **clear ()** - удаляет все элементы списка, очищая его. Никакого результата при этом не возвращается. Поддержка этого метода появилась в **Python 3.3**.

Пример:

```
arr = [1, 2, 3, 1, 1]
arr.clear ()
print(arr)
[ ]
```

Если необходимо удалить все повторяющиеся элементы списка, то можно преобразовать список во множество, а затем множество обратно преобразовать в список. Обратите внимание на то, что список должен содержать только неизменяемые объекты (например, числа, строки или кортежи). В противном случае возбуждается исключение **TypeError**.

Пример:

```
arr = [1, 2, 3, 1, 1, 2, 2, 3, 3]
s = set(arr) # Преобразуем список во множество
print(s)
{1, 2, 3}
arr = list(s) # Преобразуем множество в список
print(arr) #   Все повторы были удалены
[1, 2, 3]
```

4.8 Поиск элемента и получение сведений об элементах списка

Выполнить проверку на вхождение элемента в список позволяет оператор **in**: если элемент входит в список, то возвращается значение **True**, в противном случае – **False**. Аналогичный оператор **not in** выполняет проверку на невхождение элемента в список: если элемент отсутствует в списке, возвращается **True**, в противном случае – **False**. Примеры:

```
print(2 in [1, 2, 3, 4, 5], 6 in [1, 2, 3, 4, 5]) # Проверка на вхождение
(True, False)
print(2 not in [1, 2, 3, 4, 5] , 6 not in [1, 2, 3, 4, 5]) # Проверка на невхождение
(False, True)
```

Тем не менее, оба этих оператора не дают никакой информации о местонахождении элемента внутри списка. Чтобы узнать индекс элемента внутри списка, следует воспользоваться методом **index ()**. Формат метода:

index(<Значение>[, <Начало>[, <Конец>]])

Метод **index ()** возвращает индекс элемента, имеющего указанное значение. Если значение не входит в список, то возбуждается исключение **ValueError**. Если второй и третий параметры не указаны, то поиск будет производиться с начала и до конца списка. Пример:

```
arr = [1, 2, 1, 2, 1]
print(arr.index(1), arr.index(2))
(0, 1)
print(arr.index(1, 1), arr.index(1, 3, 5))
(2, 4)
print(arr.index (3))
Traceback (most  recent call  last):
File  "<pyshell#16>",  line 1, in <module>
```

```
arr.index(3)
ValueError: 3 is not in list
```

Узнать общее количество элементов с указанным значением позволяет метод **count (<Значение>)**. Если элемент не входит в список, то возвращается значение 0, Пример:

```
arr =[1, 2, 1, 2, 1]
print(arr.count(1), arr.count(2))
(3, 2)
print(arr.count(3)) # Элемент не входит  в список
0
```

С помощью функций **max ()** и **min ()** можно узнать максимальное и минимальное значение списка соответственно. Пример:

```
arr = [1, 2, 3, 4, 5]
print(max(arr), min(arr))
(5, 1)
```

Функция **any(<Последовательность>)** возвращает значение **True**, если в последовательности существует хоть один элемент, который в логическом контексте возвращает значение **True**. Если последовательность не содержит элементов, возвращается значение **False**. Пример:

```
print(any([0, None]), any([0, None, 1]), any([]))
(False, True, False)
```

Функция **all(<Последовательность>)** возвращает значение **True**, если все элементы последовательности в логическом контексте возвращают значение **True** или последовательность не содержит элементов. Пример:

```
print(all([0, None]), all([0, None, 1]), all([]), all (["str", 10]))
(False, False, True, True)
```

4.9 Переворачивание и перемешивание списка

Метод **reverse ()** изменяет порядок следования элементов списка на противоположный. Метод изменяет текущий список и ничего не возвращает. Пример:

```
arr = [1, 2, 3, 4, 5]
arr.reverse() # Изменяется текущий список
print(arr)
[5, 4, 3, 2, 1]
```

Если необходимо изменить порядок следования и получить новый список, то следует воспользоваться функцией **reversed (<*Последовательность*>)**. Функция возвращает итератор, который можно преобразовать в список с помощью функции **list()** или генератора списков:

```
arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
reversed(arr)
<list_reverseiterator object at 0x00FD515Q>
list (reversed(arr)) # Использование функции list()
[10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
```

```
for i in reversed(arr); print (i, end=" ") # Вывод с помощью цикла
10 9 8 7 6 5 4 3 2 1
 [i for i in reversed(arr)] # Использование генератора списков
[10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
```

Функция **shuffle (<*Список*>[, <*Число от* 0.0 *до* 1.0>])** из модуля **random** "перемешивает" список случайным образом. Функция перемешивает сам список и ничего не возвращает.

Если второй параметр не указан, то используется значение, возвращаемое функцией **random ()**. Пример:

```
import random # Подключаем модуль random
arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
random.shuffle(arr) # Перемешиваем список случайным образом
print(arr)
[2, 1, 10, 4, 6, 8, 9, 3, 1, 5]
```

4.10 Заполнение списка числами

Создать список, содержащий диапазон чисел, можно с помощью функции **range ()**. Эта функция возвращает диапазон, который преобразуется в список вызовом функции **list ()**.

Функция **range ()** имеет следующий формат:

```
range([<Начало>, ]<Конец>[, <Шаг>])
```

Первый параметр задает начальное значение - если он не указан, используется значение 0. Во втором параметре указывается конечное значение. Следует заметить, что это значение не входит в возвращаемый диапазон. Если параметр ***<Шаг>*** не указан, то используется значение 1. В качестве примера заполним список числами от 0 до 10:

```
list(range(11))
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Создадим список, состоящий из диапазона чисел от 1 до 15:

```
list(range(1, 16))
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
```

Теперь изменим порядок следования чисел на противоположный:

```
list(range(15, 0, -1))
[15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
```

Если необходимо получить список со случайными числами (или случайными элементами из другого списка), то следует воспользоваться функцией **sample (<*Последовательность*>, <*Количество элементов*>)** из модуля **random**. Пример:

```
import random
arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
random.sample(arr, 3)
[1, 9, 5]
random.sample(range(300), 5)
[259, 294, 142, 292, 245]
```

4.11 Преобразование списка в строку

Преобразовать список в строку позволяет метод **join ()**. Элементы добавляются через указанный разделитель. Формат метода:

<*Строка*> = <*Разделитель*>.join (<*Последовательность*>)

Пример:

```
arr = ["word1", "word2", "word3"]
" - ".join(arr)
'word1 - word2 - word3'
```

Обратите внимание на то, что элементы списка должны быть строками, иначе возвращается исключение **TypeError**:

```
arr = ["word1", "word2", "word3", 2]
" - ".join(arr)
Traceback (most recent call last):
File "<pyshell#69>",  line 1, in  <module>
  " - ".join(arr)
TypeError: sequence item 3: expected str instance, int found
```

Избежать этого исключения можно с помощью выражения-генератора, внутри которого текущий элемент списка преобразуется в строку с помощью функции **str ()**:

```
arr = ["word1", "word2", "word3", 2]
" - ".join((str(i) for i in arr))
'word1 - word2 - word3 - 2'
```

Кроме того, с помощью функции **str()** можно сразу получить строковое представление списка:

```
arr = ["word1", "word2", "word3", 2]
str(arr)
"['word1', 'word2', 'word3', 2]"
```

4.12 Функция map()

Встроенная функция arr = [[1, 2], [3, 4]] # Элементы имеют изменяемый тип (список)

```
for i in arr: i[0] += 10
print (arr)
```

map () позволяет применить функцию к каждому элементу последовательности. Функция имеет следующий формат:

```
map(<Функция>, <Последовательность1>[, ..., <ПоследовательностьN>])
```

Функция **map ()** возвращает объект, поддерживающий итерации (а не список, как это было ранее в **Python 2**). Чтобы получить список в версии **Python 3**, необходимо результат передать в функцию **list ()**.

В качестве параметра **<Функция>** указывается ссылка на функцию (название функции без круглых скобок), которой будет передаваться текущий элемент последовательности. Внутри функции обратного вызова необходимо вернуть новое значение. Для примера прибавим к каждому элементу списка число 10:

```
def func(elem):
    """ Увеличение значения каждого элемента списка """
    return elem + 10  #  Возвращаем новое значение
arr = [1, 2, 3, 4, 5]
print(list(map(func, arr)))
```

Результат выполнения:

```
[11, 12, 13, 14, 15]
```

Функции **map ()** можно передать несколько последовательностей. В этом случае в функцию обратного вызова будут передаваться сразу несколько элементов, расположенных в последовательностях на одинаковом смещении. Просуммируем элементы трех списков:

```
def func(e1, e2, e3):
    """ Суммирование элементов трех разных списков """
    return e1 + e2 + e3  #  Возвращаем новое значение
arr1 = [1, 2, 3, 4, 5]
arr2 = [10, 20, 30, 40, 50]
arr3 = [100, 200, 300, 400, 500]
print(list(map(func, arr1, arr2, arr3)))
```

Результат выполнения:

```
[111, 222, 333, 444, 555]
```

Если количество элементов в последовательностях будет разным, то в качестве ограничения выбирается последовательность с минимальным количеством элементов:

```
def func(e1, e2, e3):
    """ Суммирование элементов трех разных списков """
    return e1 + e2 + e3  #  Возвращаем новое значение
arr1 = [1, 2, 3, 4, 5]
arr2 = [10, 20]
arr3 = [100, 200, 300, 400, 500]
print(list(map(func, arr1, arr2, arr3)))
```

Результат выполнения:

```
[111, 222]
```

4.13 Функция zip()

Встроенная функция **zip()** на каждой итерации возвращает кортеж, содержащий элементы последовательностей, которые расположены на одинаковом смещении. Функция возвращает объект, поддерживающий итерации (а не список, как это было ранее в **Python 2**). Чтобы получить список в версии **Python 3**, необходимо результат передать в функцию **list()**.

Формат функции:

```
zip(<Последовательность1>[, ..., <ПоследовательностьN>])
```

Пример:

```
print(zip([1, 2, 3], [4, 5, 6], [7, 8, 9]))
<zip object at 0x00FCAC88>
```

```
print(list(zip([1, 2, 3], [4, 5, 6], [7, 8, 9])))
[(1, 4, 7), (2, 5, 8), (3, 6, 9)]
```

Если количество элементов в последовательностях будет разным, то в результат попадут только элементы, которые существуют во всех последовательностях на одинаковом смещении:

```
print(list(zip([1, 2, 3], [4, 6], [7, 8, 9, 10])))
[(1, 4, 7), (2, 6, 8)]
```

В качестве еще одного примера переделаем нашу программу из предыдущего шага, в которой выполняется суммирование элементов трех списков, и используем функцию **zip()** вместо функции **map()**:

```
arr1 = [1, 2, 3, 4, 5]
arr2 = [10, 20, 30, 40, 50]
arr3 = [100, 200, 300, 400, 500]
arr = [x + y + z for (x, y, z) in zip(arr1, arr2, arr3)]
print(arr)
```

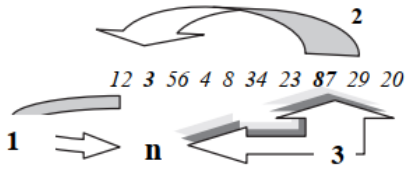
Результат выполнения:

```
[111, 222, 333, 444, 555]
```

4.14 Пример решения задачи на языке программирования Python

Задача. В списке из десяти целых чисел, сформированном случайным образом, найдите максимальный и минимальный элементы, а также осуществите их обмен.

Комментарий. Для того чтобы обменять элементы списка местами, нам потребуются две ячейки **nommax** и **nommin.** Изначально считаем, что номер максимального элемента в списке равен нулю и номер минимального элемента также равен нулю. Отсюда и операторы в программе: **nommax=0** и **nommin=0.** Приемы нахождения максимального и минимального чисел рассматривались при программировании циклических алгоритмов, поэтому сейчас мы на них останавливаться не будем. Для того чтобы осуществить обмен ячеек, необходима третья ячейка – например, **n**. Предположим, сгенерирован список целых чисел:



Минимальный элемент в списке равен **3,** с порядковым номером **1,** а максимальный элемент в списке равен **87,** с порядковым номером **7.** Возьмем третью ячейку – **n. Первым действием** (**1**) будет перемещение в нее найденного минимального числа. **Вторым действием** (**2**) на освободившееся место перемещаем максимальное число. И, наконец, **третьим действием** (**3**) минимальное число из ячейки **n** помещается на то место в списке, где только что находилось максимальное число. В программе эти действия можно было бы описать операторами:

n=chislo[1]
chislo[1]=a[7]
a[7]=n

Поскольку заранее не известно, какой элемент в списке будет минимальный, а какой максимальный, и каковы будут их номера, такую последовательность операторов заменяем на следующую:

n=chislo[nommin]
chislo[nommin]=chislo[nommax]
chislo[nommax]=n

Решение задачи:

```
import random
chislo=random.sample(range(100),10)
print(chislo)
nommin = 0 #Считаем, что номер минимального элемента в списке равен нулю
nommax = 0 #Считаем, что номер максимального элемента в списке равен нулю
min = 32767
max = -32768
for i in range(1,10):
    if chislo[i]<min: #Поиск минимального элемента в списке
        min = chislo[i]
        nommin = i #Нахождение его номера
    if chislo[i]>max: #Поиск максимального элемента в списке
        max = chislo[i]
        nommax = i #Нахождение его номера
print("Минимальное число =" , min)
print("Максимальное число =" , max)
n = chislo[nommin] #Обмен элементов списка
chislo[nommin] = chislo[nommax]
chislo[nommax] = n
for i in range(0,10):
    print(chislo[i], end='\t')
```

5 Индивидуальное задание

5.1 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 Список из N элементов заполните случайными целыми числами, находящимися в интервале от –20 до 40. Выведите на экран компьютера созданный список. В списке положительные элементы уменьшите вдвое, а отрицательные замените на значения их индексов.

2 Список из N элементов заполните случайными целыми числами, находящимися в интервале от 1 до 40. Определите, сколько процентов всего количества элементов списка составляют нечетные элементы.

3 Список из N элементов заполните случайными целыми числами, находящимися в интервале от 1 до 50. Выведите на экран компьютера созданный список и упорядочите элементы данного списка по возрастанию их значений.

4 Из списка произвольных чисел A[10] сформируйте другой список таким образом, чтобы вначале шли отрицательные элементы исходного списка, затем положительные и, наконец, нулевые.

5 Список из N элементов заполните случайными целыми числами, находящимися в интервале от 1 до 30. Выведите на экран компьютера созданный список и найдите максимальный элемент, его номер и поменяйте местами максимальный и первый элемент списка.

6 Включите заданное число D в список A[10], упорядоченный по возрастанию, с сохранением упорядоченности.

7 Из списка A, состоящего из n элементов, удалите первые четыре нулевых элемента.

8 Список из N элементов заполните случайными целыми числами, находящимися в интервале от –30 до 50. Выведите на экран компьютера созданный список и определите, есть ли в нем серии элементов, состоящих из знакочередующихся чисел. Если есть, то выведите на экран количество таких серий.

9 Из списка произвольных чисел A[10] сформируйте два списка, содержащих номера положительных и отрицательных элементов.

10 Список из N элементов заполните случайными целыми числами, находящимися в интервале от –20 до 40. Выведите на экран компьютера созданный список и вычислите среднее арифметическое значение квадратов положительных элементов.

11 Список из N элементов заполните случайными вещественными числами, находящимися в интервале от 1 до 30. Все элементы списка с четными номерами, предшествующие первому по порядку элементу с наибольшим значением, домножьте на него.

12 Список из N элементов заполните случайными целыми числами, находящимися в интервале от –15 до 20. Выведите на экран компьютера созданный список и найдите наибольший элемент из отрицательных.

13 Список из N элементов заполните случайными целыми числами, находящимися в интервале от 1 до 50. Выведите на экран компьютера созданный список и найдите количество тех элементов, значения которых находятся в диапазоне от A до B.

14 Пользователь вводит с клавиатуры элементы списка A[n]. Определите, является ли заданная последовательность чисел a1 , a2, ... , aN монотонно убывающей.

15 Список из N элементов заполните случайными целыми числами, находящимися в интервале от 1 до 30. Замените нулями элементы между максимальным и минимальным значениями, кроме них самих.

5.2 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 Список из N элементов заполните случайными целыми числами, находящимися в интервале от 1 до 50. Выведите на экран компьютера созданный список и найдите максимальный и минимальный элементы, вычислите их разность.

2 Список из N элементов заполните случайными целыми числами, находящимися в интервале от –50 до 50. Выведите на экран компьютера созданный список и найдите сумму всех элементов, расположенных после последнего отрицательного элемента.

3 Список из N элементов заполните случайными целыми числами, находящимися в интервале от 0 до 100. Определите количество элементов, равных среднему арифметическому всех элементов списка.

4 Список из N элементов заполните случайными целыми числами, находящимися в интервале от –25 до 25. Выведите на экран компьютера созданный список и замените все отрицательные элементы их модулями.

5 Список из N элементов заполните случайными целыми числами, находящимися в интервале от 1 до 60. Найдите второй по величине элемент списка.

6 Список из N элементов заполните случайными целыми числами, находящимися в интервале от –40 до 40. Выведите на экран компьютера созданный список и подсчитайте количество элементов, кратных 5.

7 Пользователь вводит с клавиатуры элементы списка A[n]. Определите, содержит ли список одинаковые элементы.

8 Список из N элементов заполните случайными целыми числами, находящимися в интервале от 10 до 99. Выведите на экран компьютера созданный список и найдите сумму цифр каждого элемента списка.

9 Список из N элементов заполните случайными целыми числами, находящимися в интервале от –30 до 30. Выведите на экран компьютера созданный список и поменяйте местами минимальный и последний элементы списка.

10 Список из N элементов заполните случайными целыми числами, находящимися в интервале от 1 до 20. Определите, сколько раз в списке встречается максимальный элемент.

11 Список из N элементов заполните случайными вещественными числами, находящимися в интервале от –10 до 10. Выведите на экран компьютера созданный список и найдите произведение положительных элементов.

12 Список из N элементов заполните случайными целыми числами, находящимися в интервале от 1 до 100. Выведите на экран компьютера созданный список и сформируйте новый список, содержащий только двузначные числа.

13 Список из N элементов заполните случайными целыми числами, находящимися в интервале от –50 до 50. Выведите на экран компьютера созданный список и найдите длину самой длинной серии положительных элементов.

14 Список из N элементов заполните случайными целыми числами, находящимися в интервале от 1 до 30. Выведите на экран компьютера созданный список и удалите из него все элементы, равные минимальному значению.

15 Список из N элементов заполните случайными целыми числами, находящимися в интервале от –20 до 20. Сформируйте новый список, содержащий квадраты только положительных элементов исходного списка.

6 Содержание отчета (отчет по лабораторной работе выполняется в электронном виде в папке учащегося)

6.1 Тема лабораторной работы
6.2 Цель лабораторной работы
6.3 Индивидуальное задание (текст программы и скриншоты выполнения)

7 Контрольные вопросы

7.1 Опишите способы создания списков в языке программирования Python.
7.2 Перечислите операции над списками в языке программирования Python.
7.3 Охарактеризуйте добавление и удаление элементов списка в языке программирования Python.
7.4 Приведите синтаксис сортировки списков в языке программирования Python.

Список используемых источников

1 Постолит, А. В. Python, Django и PyCharm для начинающих / А. В. Постолит. – СПб.: БХВ-Петербург, 2021. – 464 с.: ил.

2 Прохоренок, Н. А. Python 3. Самое необходимое / Н. А. Прохоренок, В. А. Дронов. – 2-е изд., перераб. и доп. – СПб.: БХВ-Петербург, 2019. – 608 с.: ил.

3 Фоминых, Е. И. Инструментальное программное обеспечение: учебное пособие / Е. И. Фоминых, Т. Е. Фоминых. – Минск : РИПО, 2022. – 410 с.: ил.